

EATSFOOD - LTDA

Tecnologia que alimenta resultados

Guia de Implementação - API de Parceiros

Versão 1.0.0 | Novembro 2025

Implementação Técnica - API de Parceiros

EATSFOOD - LTDA

Tecnologia que alimenta resultados

□ Arquitetura do Sistema

```
????????????????????      ?????????????????????      ?????????????????????
? Sistema do      ? HTTPS ? EATSFOOD API      ? HTTPS ? Sistema do      ?
? Parceiro A      ? ?????? ? Gateway      ? ?????? ? Parceiro B      ?
????????????????????      ?????????????????????      ?????????????????????
                        ?
                        ?????????????????????
                        ? Rate Limiter ?
                        ?????????????????????
                        ?????????????????????
                        ? Auth Middleware?
                        ?????????????????????
                        ?????????????????????
                        ? Route Handler ?
                        ?????????????????????
                        ?????????????????????
                        ? Business Logic ?
                        ?????????????????????
                        ?????????????????????
                        ? Database      ?
                        ?????????????????????
```

📁 Estrutura de Arquivos

```
app/
??? Http/
?   ??? Controllers/
?   ?   ??? Api/
?   ?   ??? Partner/
?   ?       ??? ProductController.php
?   ?       ??? OrderController.php
?   ?       ??? WebhookController.php
?   ??? Middleware/
?   ?   ??? PartnerApiAuth.php
?   ?   ??? PartnerRateLimit.php
?   ??? Requests/
?   ??? Partner/
?       ??? SyncProductsRequest.php
?       ??? UpdateOrderStatusRequest.php
??? Models/
?   ??? PartnerCredential.php
?   ??? PartnerProduct.php
?   ??? PartnerWebhook.php
?   ??? PartnerLog.php
??? Services/
?   ??? PartnerAuthService.php
?   ??? PartnerProductService.php
?   ??? PartnerOrderService.php
?   ??? WebhookDispatcher.php
??? Jobs/
?   ??? SendOrderWebhook.php
?   ??? RetryFailedWebhook.php

database/migrations/
??? 2025_11_24_create_partner_credentials_table.php
??? 2025_11_24_create_partner_products_table.php
??? 2025_11_24_create_partner_webhooks_table.php
??? 2025_11_24_create_partner_logs_table.php

routes/
??? api_partner.php
```

📦 Migrations (Database)

1. Partner Credentials

```
// 2025_11_24_create_partner_credentials_table.php
Schema::create('partner_credentials', function (Blueprint $table) {
    $table->id();
    $table->string('tenant_code', 50)->unique();
    $table->string('api_key', 64)->unique();
    $table->string('api_secret', 64);
    $table->boolean('active')->default(true);
    $table->json('allowed_ips')->nullable();
    $table->integer('rate_limit_per_minute')->default(100);
    $table->timestamp('last_used_at')->nullable();
    $table->timestamps();

    $table->index('api_key');
    $table->index('tenant_code');
});
```

2. Partner Products

```
// 2025_11_24_create_partner_products_table.php
Schema::create('partner_products', function (Blueprint $table) {
    $table->id();
    $table->string('tenant_code', 50);
    $table->string('external_id')->index();
    $table->unsignedBigInteger('produto_id')->nullable();
    $table->string('name');
    $table->text('description')->nullable();
    $table->decimal('price', 10, 2);
    $table->string('category')->nullable();
    $table->string('image_url')->nullable();
    $table->boolean('available')->default(true);
    $table->integer('preparation_time')->default(0);
    $table->timestamp('synced_at');
    $table->timestamps();

    $table->unique(['tenant_code', 'external_id']);
    $table->foreign('tenant_code')->references('tenant_code')->on('empresas');
    $table->foreign('produto_id')->references('id')->on('produtos')->onDelete('set null');
});
```

3. Partner Webhooks

```
// 2025_11_24_create_partner_webhooks_table.php
Schema::create('partner_webhooks', function (Blueprint $table) {
    $table->id();
    $table->string('tenant_code', 50);
    $table->string('url');
    $table->json('events');
    $table->boolean('active')->default(true);
    $table->integer('retry_count')->default(0);
    $table->timestamp('last_success_at')->nullable();
    $table->timestamp('last_failure_at')->nullable();
    $table->text('last_error')->nullable();
    $table->timestamps();

    $table->foreign('tenant_code')->references('tenant_code')->on('empresas');
});
```

4. Partner Logs

```
// 2025_11_24_create_partner_logs_table.php
Schema::create('partner_logs', function (Blueprint $table) {
    $table->id();
    $table->uuid('request_id')->unique();
    $table->string('tenant_code', 50);
    $table->string('method', 10);
    $table->string('endpoint');
    $table->integer('status_code');
    $table->integer('response_time_ms');
    $table->ipAddress('ip_address');
    $table->text('request_body')->nullable();
    $table->text('response_body')->nullable();
    $table->timestamp('created_at');

    $table->index(['tenant_code', 'created_at']);
    $table->index('request_id');
});
```

Middleware de Autenticação

```
// app/Http/Middleware/PartnerApiAuth.php
<?php

namespace App\Http\Middleware;

use Closure;
use Illuminate\Http\Request;
use App\Models\PartnerCredential;
use App\Services\PartnerAuthService;

class PartnerApiAuth
{
    protected $authService;

    public function __construct(PartnerAuthService $authService)
    {
        $this->authService = $authService;
    }

    public function handle(Request $request, Closure $next)
    {
        $apiKey = $request->bearerToken();
        $tenantCode = $request->header('X-Tenant-Code');
        $signature = $request->header('X-Signature');

        if (!$apiKey || !$tenantCode || !$signature) {
            return response()->json([
                'success' => false,
                'error' => [
                    'code' => 'MISSING_CREDENTIALS',
                    'message' => 'API Key, Tenant Code e Signature são obrigatórios'
                ]
            ], 401);
        }

        // Buscar credenciais
        $credentials = PartnerCredential::where('api_key', $apiKey)
            ->where('tenant_code', $tenantCode)
            ->where('active', true)
            ->first();

        if (!$credentials) {
            return response()->json([
                'success' => false,
                'error' => [
                    'code' => 'INVALID_CREDENTIALS',
                    'message' => 'Credenciais inválidas'
                ]
            ], 401);
        }

        // Validar assinatura HMAC
        $requestBody = $request->getContent();
        $expectedSignature = hash_hmac('sha256', $requestBody, $credentials->api_secret);

        if (!hash_equals($expectedSignature, $signature)) {
            return response()->json([
                'success' => false,
                'error' => [
                    'code' => 'INVALID_SIGNATURE',
                    'message' => 'Assinatura HMAC inválida'
                ]
            ], 401);
        }
    }
}
```

```
}

// Validar IP (se whitelist configurada)
if ($credentials->allowed_ips) {
    $allowedIps = json_decode($credentials->allowed_ips, true);
    $clientIp = $request->ip();

    if (!in_array($clientIp, $allowedIps)) {
        return response()->json([
            'success' => false,
            'error' => [
                'code' => 'IP_NOT_ALLOWED',
                'message' => 'IP não autorizado'
            ]
        ], 403);
    }
}

// Atualizar último uso
$credentials->update(['last_used_at' => now()]);

// Adicionar credenciais ao request
$request->merge(['partner_credentials' => $credentials]);

return $next($request);
}
```

□□ Controllers

Product Controller

```
// app/Http/Controllers/Api/Partner/ProductController.php
<?php

namespace App\Http\Controllers\Api\Partner;

use App\Http\Controllers\Controller;
use App\Http\Requests\Partner\SyncProductsRequest;
use App\Services\PartnerProductService;
use Illuminate\Http\Request;

class ProductController extends Controller
{
    protected $productService;

    public function __construct(PartnerProductService $productService)
    {
        $this->productService = $productService;
    }

    public function sync(SyncProductsRequest $request)
    {
        $tenantCode = $request->partner_credentials->tenant_code;
        $products = $request->input('products');

        $result = $this->productService->syncProducts($tenantCode, $products);

        return response()->json([
            'success' => true,
            'message' => 'Produtos sincronizados com sucesso',
            'data' => $result
        ]);
    }

    public function updateAvailability(Request $request, $externalId)
    {
        $tenantCode = $request->partner_credentials->tenant_code;
        $available = $request->input('available');
        $reason = $request->input('reason');

        $this->productService->updateAvailability(
            $tenantCode,
            $externalId,
            $available,
            $reason
        );

        return response()->json([
            'success' => true,
            'message' => 'Disponibilidade atualizada',
            'data' => [
                'external_id' => $externalId,
                'available' => $available
            ]
        ]);
    }
}
```


Order Controller

```
// app/Http/Controllers/Api/Partner/OrderController.php
<?php

namespace App\Http\Controllers\Api\Partner;

use App\Http\Controllers\Controller;
use App\Services\PartnerOrderService;
use Illuminate\Http\Request;

class OrderController extends Controller
{
    protected $orderService;

    public function __construct(PartnerOrderService $orderService)
    {
        $this->orderService = $orderService;
    }

    public function index(Request $request)
    {
        $tenantCode = $request->partner_credentials->tenant_code;

        $orders = $this->orderService->listOrders($tenantCode, [
            'status' => $request->input('status'),
            'date_from' => $request->input('date_from'),
            'date_to' => $request->input('date_to'),
            'page' => $request->input('page', 1),
            'per_page' => min($request->input('per_page', 20), 100)
        ]);

        return response()->json([
            'success' => true,
            'data' => $orders
        ]);
    }

    public function updateStatus(Request $request, $orderId)
    {
        $tenantCode = $request->partner_credentials->tenant_code;

        $request->validate([
            'status' => 'required|in:confirmed,preparing,ready,out_for_delivery,delivered,cancelled',
            'message' => 'nullable|string',
            'estimated_ready_time' => 'nullable|date'
        ]);

        $order = $this->orderService->updateStatus(
            $tenantCode,
            $orderId,
            $request->input('status'),
            $request->input('message'),
            $request->input('estimated_ready_time')
        );

        return response()->json([
            'success' => true,
            'message' => 'Status atualizado',
            'data' => $order
        ]);
    }
}
```

Webhook Dispatcher

```
// app/Services/WebhookDispatcher.php
<?php

namespace App\Services;

use App\Models\PartnerWebhook;
use App\Jobs\SendOrderWebhook;
use Illuminate\Support\Str;

class WebhookDispatcher
{
    public function dispatchOrderCreated($pedido)
    {
        $tenantCode = $pedido->empresa->tenant_code;

        $webhook = PartnerWebhook::where('tenant_code', $tenantCode)
            ->where('active', true)
            ->whereJsonContains('events', 'order.created')
            ->first();

        if (!$webhook) {
            return;
        }

        $payload = $this->buildOrderPayload($pedido, 'order.created');

        SendOrderWebhook::dispatch($webhook, $payload);
    }

    protected function buildOrderPayload($pedido, $event)
    {
        $timestamp = now()->toIso8601String();

        $payload = [
            'event' => $event,
            'timestamp' => $timestamp,
            'signature' => '', // Será preenchido no Job
            'data' => [
                'order_id' => 'ORD-' . $pedido->id,
                'platform_order_id' => $pedido->id,
                'customer' => [
                    'name' => $pedido->cliente->nome,
                    'phone' => $pedido->cliente->telefone,
                    'email' => $pedido->cliente->email
                ],
                'delivery_address' => [
                    'street' => $pedido->delivery->endereco_rua,
                    'number' => $pedido->delivery->endereco_numero,
                    'complement' => $pedido->delivery->endereco_complemento,
                    'neighborhood' => $pedido->delivery->endereco_bairro,
                    'city' => $pedido->delivery->endereco_cidade,
                    'state' => $pedido->delivery->endereco_estado,
                    'zipcode' => $pedido->delivery->endereco_cep,
                    'reference' => $pedido->delivery->ponto_referencia
                ],
                'items' => $pedido->itens->map(function($item) {
                    $partnerProduct = $item->produto->partnerProduct;
                    return [
                        'external_id' => $partnerProduct->external_id ?? null,
                        'name' => $item->produto->nome,
                        'quantity' => $item->quantidade,
                        'unit_price' => (float) $item->preco_unitario,
                        'subtotal' => (float) $item->subtotal,
```

```
        'notes' => $item->observacoes
    ];
    })->toArray(),
    'payment' => [
        'method' => $pedido->forma_pagamento,
        'status' => $pedido->status_pagamento,
        'amount' => (float) $pedido->total
    ],
    'totals' => [
        'subtotal' => (float) $pedido->subtotal,
        'delivery_fee' => (float) $pedido->taxa_entrega,
        'service_fee' => (float) $pedido->taxa_servico,
        'total' => (float) $pedido->total
    ],
    'expected_delivery_time' => now()->addMinutes($pedido->tempo_entrega)->toIso8601String()
    ]
];

return $payload;
}
}
```

Job de Envio de Webhook

```
// app/Jobs/SendOrderWebhook.php
<?php

namespace App\Jobs;

use App\Models\PartnerWebhook;
use Illuminate\Bus\Queueable;
use Illuminate\Contracts\Queue\ShouldQueue;
use Illuminate\Foundation\Bus\Dispatchable;
use Illuminate\Queue\InteractsWithQueue;
use Illuminate\Queue\SerializesModels;
use Illuminate\Support\Facades\Http;
use Illuminate\Support\Facades\Log;

class SendOrderWebhook implements ShouldQueue
{
    use Dispatchable, InteractsWithQueue, Queueable, SerializesModels;

    public $tries = 5;
    public $backoff = [60, 300, 900, 3600]; // 1min, 5min, 15min, 1h

    protected $webhook;
    protected $payload;

    public function __construct(PartnerWebhook $webhook, array $payload)
    {
        $this->webhook = $webhook;
        $this->payload = $payload;
    }

    public function handle()
    {
        // Gerar assinatura HMAC
        $payloadJson = json_encode($this->payload['data']);
        $credentials = $this->webhook->partnerCredential;
        $signature = hash_hmac('sha256', $payloadJson, $credentials->api_secret);

        $this->payload['signature'] = $signature;

        try {
            $response = Http::timeout(30)
                ->withHeaders([
                    'X-Webhook-Signature' => $signature,
                    'X-Webhook-Timestamp' => now()->timestamp,
                    'Content-Type' => 'application/json'
                ])
                ->post($this->webhook->url, $this->payload);

            if ($response->successful()) {
                $this->webhook->update([
                    'last_success_at' => now(),
                    'retry_count' => 0,
                    'last_error' => null
                ]);

                Log::info('Webhook enviado com sucesso', [
                    'webhook_id' => $this->webhook->id,
                    'event' => $this->payload['event'],
                    'order_id' => $this->payload['data']['order_id']
                ]);

                return;
            }
        }
    }
}
```

```

        throw new \Exception('HTTP ' . $response->status() . ': ' . $response->body());
    } catch (\Exception $e) {
        $this->webhook->update([
            'last_failure_at' => now(),
            'retry_count' => $this->webhook->retry_count + 1,
            'last_error' => $e->getMessage()
        ]);

        Log::error('Falha ao enviar webhook', [
            'webhook_id' => $this->webhook->id,
            'error' => $e->getMessage(),
            'attempt' => $this->attempts()
        ]);

        if ($this->attempts() < $this->tries) {
            throw $e; // Retry
        }
    }
}
}
}

```

Rotas

```

// routes/api_partner.php
<?php

use Illuminate\Support\Facades\Route;
use App\Http\Controllers\Api\Partner\ProductController;
use App\Http\Controllers\Api\Partner\OrderController;

Route::prefix('v1/partner')->middleware(['partner.auth', 'partner.rate.limit'])->group(function () {

    // Produtos
    Route::post('/products/sync', [ProductController::class, 'sync']);
    Route::patch('/products/{external_id}/availability', [ProductController::class, 'updateAvailability']);

    // Pedidos
    Route::get('/orders', [OrderController::class, 'index']);
    Route::post('/orders/{order_id}/status', [OrderController::class, 'updateStatus']);

});

```

Configuração no `.env`

```

# API de Parceiros
PARTNER_API_ENABLED=true
PARTNER_API_RATE_LIMIT=100
PARTNER_WEBHOOK_TIMEOUT=30
PARTNER_WEBHOOK_RETRIES=5

```

Próximos passos: Implementar rate limiting, criar comando Artisan para gerar credenciais, e criar painel administrativo para gestão de parceiros.

Versão: 1.0.0

Última atualização: 24/11/2025

Empresa: EATSFOOD - LTDA

Slogan: Tecnologia que alimenta resultados

© 2025 EATSFOOD - LTDA. Todos os direitos reservados.

Produto: EATSFOOD - LTDA

Tecnologia que alimenta resultados

Empresa responsável: MYD Facilyta Technology | CNPJ: 24.223.868/0001-19

Av dos Holandeses/Cons. Hilton Rodrigues, 15, Olho D'Água, São Luís/MA

E-mail: misael_ribeiro@mydsistemas.com.br | Telefone: (98) 8848-9512

© 2025 MYD Facilyta Technology. Todos os direitos reservados.

Este documento é confidencial e destinado exclusivamente aos parceiros autorizados.